



# **EVENT MANAGEMENT SYSTEM PROJECT REPORT**

**Submitted by : Bhavya Shrishti**  
**Registration no. : 26BCE11554**  
**Course : Introduction to Problem  
Solving and Programming**  
**Platform : VITyarthi - Build Your Own  
Project**

## 1. Introduction

The Event Management System is a python desktop application with a Tkinter graphical interface that lets users record and keep track of upcoming events in one place. Meetings, birthdays, workshops, seminars and parties are often noted in chats, notebooks or sticky notes and then forgotten. This program gives students, club coordinators and working professionals a simple, private list of their events, sorted by date, that works without an internet connection, an online calendar account or any extra libraries. All data is stored locally on the user's own computer.

## 2. Problem Statement

### The Problem:

Events are usually noted in scattered places, so they are easily forgotten or double-booked. Most calendar and event tools need an online account and syncing, which is more than a user needs when they only want a quick local list of what is coming up.

### The Solution:

A Python program with a simple window where the user fills in a form with an event's name, date, category and venue. The program checks the input, keeps the events sorted by date, shows how many are still upcoming, and saves everything to a local JSON file, with no internet connection or external account needed.

## 3. Functional Requirements

The system provides three major functional modules:

- **Event Entry & Validation (Input Handling):** The user enters the event name, date (YYYY-MM-DD), category and an optional venue. Blank names, wrongly formatted dates, past dates and invalid categories are rejected with an error message.
- **Event List & Display (Graphical Interface):** Events are shown in a table sorted by date, with past events in grey and a counter for total and upcoming events. The user deletes an event by selecting its row and confirming.
- **Storage (Data Persistence):** Every change is saved to events.json and the saved events are loaded again when the program starts.

## 4. Non-Functional Requirements

These define how the system performs its functions:

- **Performance:** Adding or deleting an event, saving it to disk and showing the list happen instantly.

- **Reliability (Offline by Design):** The program never needs the internet, so it works the same every time. If events.json is missing or empty, it starts with an empty list instead of crashing.
- **Usability:** A one window with a form, dropdown and buttons, and Enter key to add (Add, View, Delete, Exit). Categories can be chosen by number or by typing the name in any case, e.g. "meeting" or "MEETING".
- **Error Handling:** Invalid input shows an error pop-up and the window stays open instead of stopping the program.
- **Portability:** Only Python's standard library is used, so the program runs on any computer with Python installed.
- **Maintainability:** The code is split into three small classes, so each part can be read and changed on its own.

## 5. System Architecture

The project uses an Object-Oriented Architecture with three classes, each in charge of one layer:

| Class        | Layer              | What it does                                                                                                              |
|--------------|--------------------|---------------------------------------------------------------------------------------------------------------------------|
| EventApp     | Presentation (GUI) | Builds the Tkinter window, reads form input, validates it and updates the table.                                          |
| EventManager | Logic & Data       | Keeps the list of events and loads, saves, adds, deletes and counts upcoming events using events.json.                    |
| Event        | Data model         | Stores one event's name, date, category and venue, and converts it to a dictionary for saving or a table row for display. |

EventApp uses one EventManager, and the EventManager holds many Event objects. Keeping the menu separate from the data makes the program easier to follow and debug.

## 6. Design Diagrams

### 6.1 Use Case Diagram

**Actor:** User

**Use Cases:**

- Enter Event Name
- Enter Event Date
- Select Category (number or name)
- Enter Venue (optional)

- Add Event
- View Sorted Event List
- Select and Delete Event
- View Total and Upcoming Count
- Exit Program

## 6.2 Workflow Diagram

Start -> Run Program -> Load Saved Events from JSON -> Show Window → User Clicks a Button -> (Add: Enter Details -> Validate Input -> Invalid: Show Error / Valid: Save Event and Sort by Date) / (View: Print List and Counter) / (Delete: Choose Number -> Remove and Save) -> (Exit) -> End.

## 6.3 Sequence Diagram

1. The user fills in the form and clicks Add Event.
2. EventApp asks for the name, date, category and venue.
3. If the name is blank, or the date is wrongly formatted or in the past, an error is shown in a pop-up and the program returns to the menu.
4. If valid, EventApp creates a new Event object.
5. EventApp calls EventManager.add(), which adds the event and sorts the list by date.
6. EventManager.save() writes the updated list to events.json.
7. The table and counter refresh, the form clears, and 'Event added!' pops up.

## 7. Design Decisions & Rationale

- **Language:** Python was chosen for its simple syntax and built-in support for dates, files and JSON.
- **Interface:** Tkinter was used because it comes with Python, so no extra installation is needed, and it gives a friendlier interface than a terminal menu while keeping the code short.
- **Classes:** Three small classes keep the data (Event, EventManager) separate from the menu (EventApp), and remove the need for global variables.
- **Storage:** A local JSON file was chosen instead of a database, because it is easy to read, needs no setup and keeps the program fully offline.
- **Date Format:** Dates are stored as YYYY-MM-DD text, so sorting the text also sorts the events in date order.

## 8.Screenshots

### 1. Adding an event

Event Manager

Event Manager

Event Details

Event Name:

ADI'S party

Date (YYYY-MM-DD):

2026-12-18

Category:

Birthday

Venue (optional):

garden view

Add Event

Delete Selected

Clear Form

Exit

Scheduled Events

| Date | Name | Category | Venue |
|------|------|----------|-------|
|------|------|----------|-------|

Total Events: 0 | Upcoming: 0

### 2. Viewing the event list, sorted by date

Event Manager

Event Manager

Event Details

Event Name:

Date (YYYY-MM-DD):

Category:

Other

Venue (optional):

Add Event

Delete Selected

Clear Form

Exit

Scheduled Events

| Date       | Name        | Category | Venue       |
|------------|-------------|----------|-------------|
| 2026-12-18 | ADI'S party | Birthday | garden view |

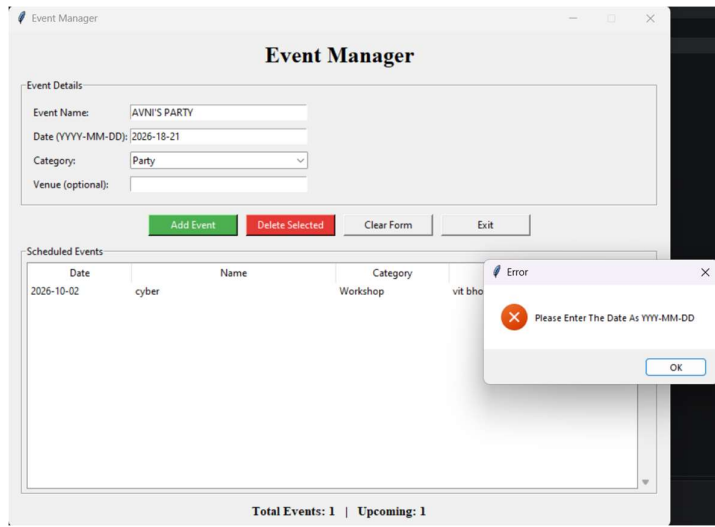
Total Events: 1 | Upcoming: 1

Success

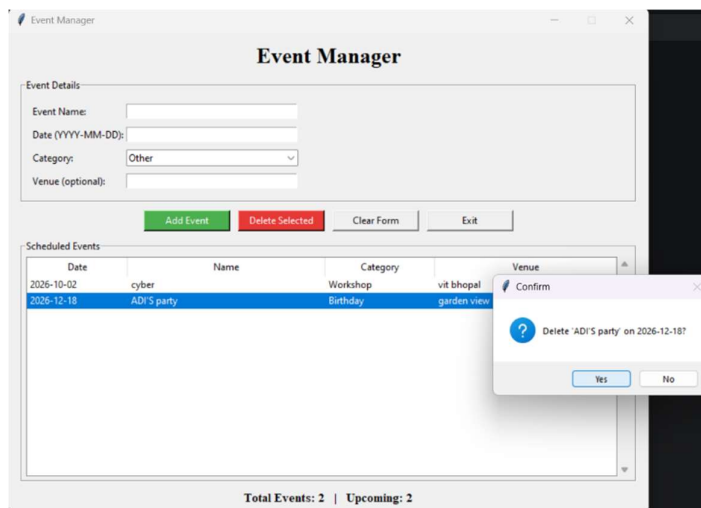
Event added!

OK

### 3. Error messages for a blank name, a wrong date format, a past date and a wrong menu choice



### 4. Deleting an event



## 9. Testing Approach

- **Unit Testing:** The EventManager class was tested on its own. Events added out of order were stored in date order, the upcoming count was correct, and deleting one left the others.
- **Boundary Testing:** Tested a blank name, a wrongly formatted date (15/12/2026), a past date, an unknown category and a clicking Delete with no row selected to make sure each shows the right message.
- **Case Testing:** Categories were entered as a number (5), in lowercase ("meeting") and in uppercase ("BIRTHDAY"), and all were saved with the correct name.

- **Persistence Testing:** The program was closed and run again after adding events to check that all events reload correctly from events.json.
- **GUI Testing:** "checked that the buttons, the Enter key and the table refresh work correctly.

## 10. Challenges Faced

- Checking dates correctly: rejecting badly formatted dates and dates that have already passed.
- Accepting a category typed in any case while still saving it with the correct name.
- Converting Event objects to dictionaries for saving and back again when loading from JSON.
- Handling a missing or empty events.json file on the very first run.
- Laying out widgets with `grid` and `pack`.
- Keeping the table in sync with the saved list after adding or deleting.

## 11. Learnings & Key Takeaways

- Learned how to use classes, constructors (`__init__`) and `self` to group related data and functions together.
- Gained practical experience with the `datetime` module for checking and comparing dates.
- Learned to read and write JSON files for simple data persistence without a database.
- Understood why separating the user interface from the data logic makes a program easier to test and change.
- Learned to build a GUI with Tkinter widgets (Entry, Combobox, Treeview, messagebox) and connect buttons to functions.

## 12. Future Enhancements

- **Reminders:** Show events that are due today or tomorrow when the program starts.
- **Edit Events:** Let the user change an event's details instead of deleting and adding it again.
- **Search and Filter:** Find events by name or show only one category.
- **Calendar date picker:** Add a simple Tkinter window on top of the same EventManager class.
- **Export:** Save the event list as a CSV file to open in Excel.

### 13. References

1. Python Documentation: <https://docs.python.org/3/>
2. Python JSON Module Documentation:  
<https://docs.python.org/3/library/json.html>
3. Python datetime Module Documentation:  
<https://docs.python.org/3/library/datetime.html>
4. Python Classes Tutorial: <https://docs.python.org/3/tutorial/classes.html>
5. Tkinter documentation: <https://docs.python.org/3/library/tkinter.html>